DOCUMENTO TÉCNICO DE ARQUITECTURA E INFRAESTRUCTURA

Porttos Móvil - Plataforma IA

DOCUMENTO TÉCNICO DE ARQUITECTURA E INFRAESTRUCTURA

Porttos Móvil (Android / iOS) - Plataforma IA

---

1. Visión General Técnica

Porttos Móvil es la solución cliente híbrida nativa de la plataforma **Porttos**, diseñada para proporcionar a directivos, analistas y operadores visibilidad operativa, alertas en tiempo real y asistencia basada en inteligencia artificial desde dispositivos móviles (teléfonos y tabletas).

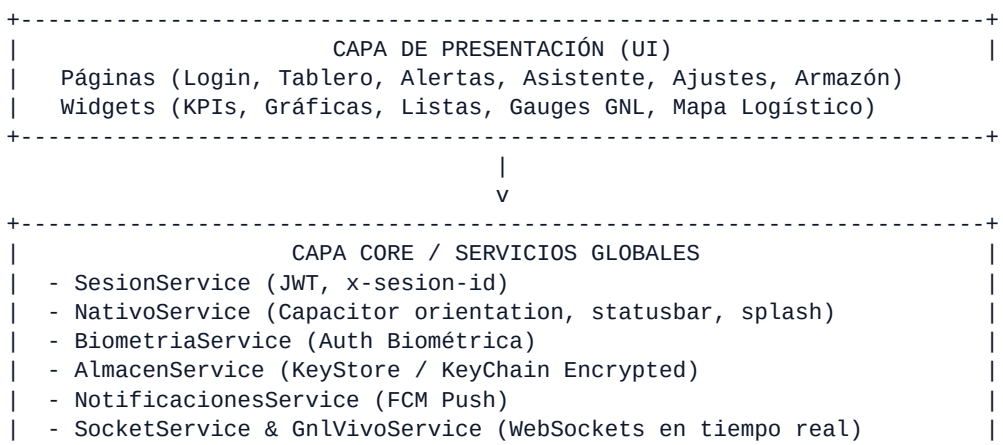
1.1 Ficha Técnica del Proyecto

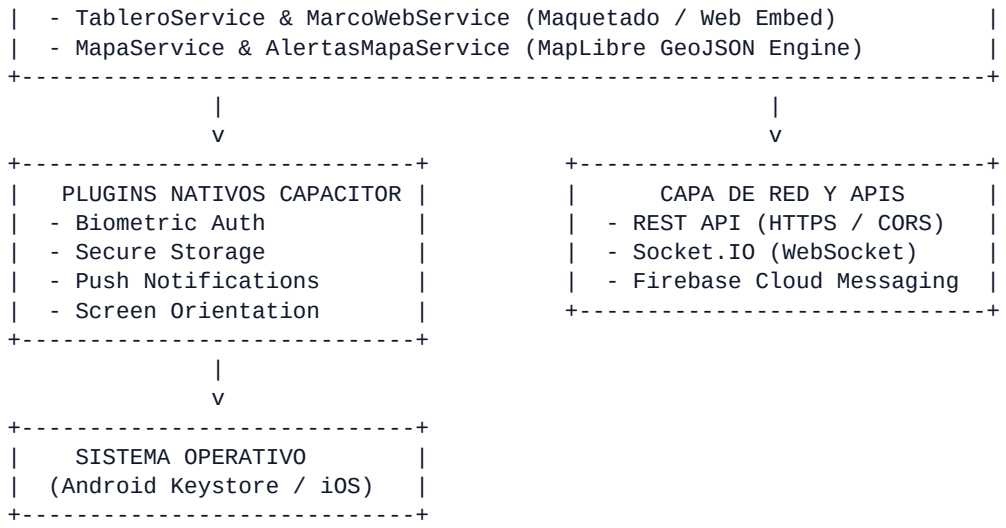
- Identificador de Aplicación (Package/App ID): `co.porttos.movil`
- Framework Frontend: Angular v17.3 (Standalone Components & Signals)
- Motor de Envolvente Nativa: Capacitor v7.0
- Lenguaje de Programación: TypeScript 5.2 / Java 17
- Librerías Visuales y Mapas: MapLibre GL v5.24, Chart.js v4.6, NgxTranslate v15.0
- Compatibilidad Android: API Level 22+ (Android 5.1+ mínimo / API 34 Target)
- Compatibilidad iOS: iOS 14.0+
- Protocolo de Red: HTTP/2 / HTTPS + WebSockets (Socket.IO v4.8)

---

2. Arquitectura de Software

La aplicación sigue un patrón de **Arquitectura Capas Abstraída y Desconectada** que garantiza desacoplamiento estricto entre la interfaz de usuario, la lógica de red y los componentes de hardware del dispositivo móvil.





2.1 Reglas Estrictas de Código

- Aislamiento de HTTP y Storage: Las páginas y componentes de UI no importan `HttpClient`, `localStorage` ni `socket.io-client`. Todo el consumo pasa por `src/app/core/`.
- Encapsulamiento de Plugins Nativo: Los plugins de Capacitor solo se referencian en los archivos correspondientes dentro de `core/` (`almacen.service.ts`, `nativo.service.ts`, `biometria.service.ts`, `notificaciones.service.ts`).
- Resolución Dinámica de Entidades: Ningún recurso operativo (puerto, terminal, brazo) está codificado en duro. La estructura se obtiene dinámicamente desde el endpoint `/api/puertos`.
- Protocolo de Sesión Única: La app escucha el evento de WebSocket `sesion-reemplazada`. Al ser expulsado por un inicio de sesión en otro dispositivo, se destruyen los tokens locales y se redirige a la vista de ingreso.

---

3. Catálogo Técnico de Servicios Core

3.1 `SesionService` (`src/app/core/sesion.service.ts`)

- Propósito: Gestiona la autenticación del usuario mediante tokens JWT.
- Cabeceras HTTP: Inyecta automáticamente el token de portador y la cabecera `x-sesion-id` mediante `SesionInterceptor`.
- Persistencia: Almacena el token y estado de sesión mediante `AlmacenService` en el Keystore/Keychain cifrado.

3.2 `BiometriaService` (`src/app/core/biometria.service.ts`)

- Librería: `@aparajita/capacitor-biometric-auth`
- Funcionalidad: Detecta disponibilidad de hardware (Huella dactilar o Face ID). Tras un login exitoso con contraseña, permite guardar credenciales de forma cifrada para futuros inicios de sesión inmediatos mediante biometría.

3.3 `AlmacenService` (`src/app/core/almacen.service.ts`)

- Librería: `@aparajita/capacitor-secure-storage`
- Funcionalidad: Almacena pares clave-valor cifrados. En Android utiliza el hardware Keystore y cifrado AES-256; en iOS utiliza el sistema Keychain.

3.4 `NotificacionesService` (`src/app/core/notificaciones.service.ts`)

- Librería: `@capacitor/push-notifications`
- Funcionalidad: Gestiona el registro del dispositivo ante Firebase Cloud Messaging (FCM). Registra el `fcm_token` en la base de datos central a través del API para la entrega dirigida de alertas operativas.

3.5 `SocketService` y `GnlVivoService` (`src/app/core/`)

- Librería: `socket.io-client`
- Funcionalidad: Establece canales WebSocket seguros con el backend. `GnlVivoService` consume la telemetría en tiempo real de los brazos y tanques de GNL, sobreponiendo valores en vivo a los widgets del tablero sin recargar la página.

3.6 `TableroService` y `MarcoWebService` (`src/app/core/`)

- Funcionalidad: Consume el endpoint `/api/movil/tablero/:puerto/:pestana`. Traduce las configuraciones de plantilla de la web (`dbo.PTLLayouts` y `dbo.PTLWidgetsMaestro`) a componentes nativos en la app. En tabletas, `MarcoWebService` permite renderizar la Torre de Control mediante un iframe optimizado con fallback automático a la vista nativa si falla la conectividad.

3.7 `MapaService` y `AlertasMapaService` (`src/app/core/`)

- Librerías: `maplibre-gl`, `maplibre-gl-js`
- Funcionalidad: Renderiza mapas vectoriales basados en MapTiler. Procesa cuatro fuentes de datos GeoJSON dinámicas: geocercas, buques/naves por estado, camiones y eventos viales.

---

4. Configuración Nativa en Android

4.1 Estructura del Manifiesto (`android/app/src/main/AndroidManifest.xml`)

- Actividad Principal: `co.porttos.movil.MainActivity` (Extiende `BridgeActivity`).
- Permisos requeridos:

```xml
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.USE_BIOMETRIC" />
```

- Provider de Archivos:

```xml
<provider
    android:name="androidx.core.content.FileProvider"
    android:authorities="${applicationId}.fileprovider"
    android:exported="false"
    android:grantUriPermissions="true">
    <meta-data
        android:name="android.support.FILE_PROVIDER_PATHS"
        android:resource="@xml/file_paths" />
</provider>
```

4.2 Configuración de Gradle (`android/app/build.gradle`)

- Namespace: `co.porttos.movil`
- Application ID: `co.porttos.movil`
- Compile SDK: `34`
- Min SDK: `22`

- Target SDK: `34`
- Versión de Gradle Wrapper: `8.7.2`

4.3 Capacitor Config (`capacitor.config.ts`)

```
import type { CapacitorConfig } from '@capacitor/cli';

const desarrollo = process.env['PORTTOS_ANDROID_DEV'] === '1';

const config: CapacitorConfig = {
    appId: 'co.porttos.movil',
    appName: 'Porttos',
    webDir: 'www',
    server: {
        androidScheme: 'https',
        cleartext: desarrollo
    },
    android: {
        allowMixedContent: desarrollo
    },
    plugins: {
        SplashScreen: { launchShowDuration: 1200, backgroundColor: '#242424', showSpinner:
false },
        StatusBar: { overlaysWebView: false, backgroundColor: '#346BA6', style: 'DARK' }
    }
};

export default config;
```

5. Seguridad y Comunicaciones

- Requisitos de CORS en API Backend:

El servidor backend debe incluir en `CORS_ORIGINS` los esquemas originados por Capacitor: `https://localhost` (Android) y `capacitor://localhost` (iOS).

- Cifrado de Credenciales:

Las credenciales de acceso para autologin biométrico se encriptan con algoritmos nativos AES-256 en el Keystore del dispositivo y requieren autorización biométrica previa para su desencriptación.

- Control de Inyección y Contenido:

Las respuestas textuales entregadas por el Asistente IA son procesadas mediante la función `markdownSeguro()` que sanitiza y escapa cualquier entidad HTML maliciosa antes del renderizado en el DOM.

6. Procedimientos de Compilación y Despliegue

6.1 Requisitos del Entorno de Desarrollo

- Node.js LTS (v20.x recomendado)
- Angular CLI (`npm i -g @angular/cli@17`)
- Android Studio Ladybug (2024.2.1+) con SDK 34 y JDK 17
- Xcode 16+ (exclusivo para generación de builds iOS en macOS)

6.2 Comandos de Ejecución

```
# Instalación de dependencias
npm install
```

```
# Servidor web local de desarrollo
npm start

# Verificación de arquitectura y pruebas automáticas
npm run test:core

# Construcción de producción y sincronización con plataformas nativas
npm run cap:sync

# Apertura del proyecto Android en Android Studio (modo producción)
npm run android

# Apertura del proyecto Android con bandera de desarrollo local HTTP
npm run android:dev
```